AtCoder Beginner Contest Discord Reminder Bot 詳細設計書

1. 文書情報

項目	内容
文書名	AtCoder Beginner Contest Discord Reminder Bot 詳細設計書
対応する要件定義書	docs/atcoder_abc_discord_reminder_requirements.md
改訂日	2026-10-10
改訂内容	ABC終了後の次回日時通知を追加（メンションなし）。土曜の太字未開催通知とWebhook専用方式を維持
実行機体	Ubuntu Serverを搭載したDynabookパソコン（SSHホスト: ryutoserver）
配置先	/srv/shared/remider（Windows側のY:\remiderに対応）
ソース管理場所	本リポジトリの開発ディレクトリ
基本言語	C++17
通知方式	Discord Webhook REST API（Gateway不使用）
外部サーバー	使用しない
月額固定費	増加させない

2. 目的と設計方針

AtCoder Beginner Contest（ABC）の次回開催日時をAtCoder公式ページから自動取得し、開催2時間前にDiscordへ通知する。通知先は固定のDiscordチャンネルとし、Ubuntu Serverを搭載したDynabookを常時稼働させて実行する。

今回の変更により、要件定義書に残るESP32固有の制約は次のように置き換える。

旧方針	新方針
ESP32単体	Ubuntu Serverを搭載したDynabook上の常駐サービス
TypeScript + Moddable SDK	C++17 + CMake
Arduino Preferences / NVS	SQLite3によるローカル永続化
ESP32向け軽量化・DOMなし解析	PCのメモリを利用したlibxml2 HTML解析
Wi-Fi接続状態の確認	対象外。OSが提供するネットワークを利用し、接続確認処理は持たない
ESP32再起動復元	systemd再起動・OS再起動後のSQLite復元

2.1 優先順位

1. 誤ったコンテスト日時を通知しない
2. 通知失敗後に再送できる
3. プロセス再起動後も通知状態を復元できる
4. 同一通知の重複を抑止する

5. AtCoderへのアクセスを必要最小限にする

6. 実行速度を最適化する

実行速度より確実な動作を優先するため、HTML解析と状態保存には実績のある外部ライブラリ・SQLiteを使用する。HTMLを推測で解析できたことにしたり、保存に失敗した状態を通知済みにしたりしない。

2.2 初期リリースの範囲

含める機能は次のとおりとする。

- ABCのみの自動検出

- 次回ABCの選択

- 開始2時間前のDiscord通知

- ABC終了後に次回の開催日時を通知（メンションなし）

- 毎週土曜日19時（JST）に当日のABC開催予定を確認し、ない場合にDiscord通知

- 通知本文全体の太字表示

- 開催日時変更の検知

- 通知後に日時が変更された場合の訂正通知

- 通知済み状態のSQLite保存

- systemdによる自動起動・異常終了時の再起動

- Discord Gatewayを使わないREST通知

初期リリースにはARC、AHC、Web設定画面、Discordからの設定変更、OTA、クラウドサーバー、外部DB、外部監視サービスを含めない。

2.3 ソースとデプロイ先の分離

ソースコードは本リポジトリの開発ディレクトリで管理し、実行時に必要な成果物だけをSSH経由でサーバーへデプロイする。開発用ソース、テスト、ビルド中間ファイルを実行環境へ直接配置しない。

```
開発環境
  └── 本リポジトリ
      ├── C++ソース
      ├── テスト
      └── ビルド成果物
            |
            | SSH/SCPまたはrsync
            v
SSH接続先 ryutoserver
  └── /srv/shared/remider
      ├── bin/atcoder-abc-reminder
      ├── state.db
      └── release-info.txt
```

ソースから生成した実行ファイルは一時ファイルへ転送し、ハッシュ値と実行権限を確認してから配置する。systemdの再起動またはサービス再読込は、転送・検証が成功した後に実行する。

3. 通知本文仕様

3.1 Discordへ送る本文

Discord APIへ送るcontentは次の形式に固定する。{ROLE_ID}は競プロロールのID、{N}はABC番号、{START}と{END}はJSTのHH:mmで置換する。<@&{ROLE_ID}>はDiscordのロールメンションで、表示上は@競プロになる。先頭からロールメンション、空行、本文の順に配置し、Discord Markdownの**で各行を太字にする。本文内の#はDiscord Markdownの見出し、[...]はコンテストURLへのインラインリンクである。

```
**<@&{ROLE_ID}>**

# **AtCoder Beginner Contest {N}**

**本日 {START} ～ {END} に [AtCoder Beginner Contest {N}](https://atcoder.jp/contests/abc{N}) が開催されます。**

**皆さんぜひ参加しましょう！□**
```

例:

```
**<@&123456789012345678>**

# **AtCoder Beginner Contest 475**

**本日 21:00 ～ 22:40 に [AtCoder Beginner Contest 475](https://atcoder.jp/contests/abc475) が開催されます。**

**皆さんぜひ参加しましょう！□**
```

3.2 土曜日のABC未開催通知

毎週土曜日19:00（JST）以降にAtCoder予定表を取得し、開始日時が当日（JST）のABCがあるか確認する。予定・開催中・直近終了の一覧を対象とし、当日の早い時刻に開催済みのABCも未開催と判定しない。当日のABCがある場合は確認済みとして記録し、追加通知は送らない。ない場合は設定済みロールへ次の通知を送る。ロールメンションと本文は太字で表示する。

```
**<@&{ROLE_ID}>**

**今日ABCは開催されません**
```

確認日ごとにSQLiteへ完了状態を保存し、同じ日の通知を重複させない。AtCoder取得に失敗した場合は通常の取得バックオフ（5分から最大1時間）で再試行し、Discord送信に失敗した場合は5分後に再試行する。いずれも成功するまで完了扱いにしない。19時以降にサービスが起動した場合も、その日の確認が未完了なら実行する。

3.3 Discordリンクプレビュー

本文中のAtCoderインラインリンクにより、Discordの標準リンクプレビューを有効にする。画像に表示されているAtCoderのタイトル、説明、ロゴはAtCoderページのOGP情報をDiscordが取得して表示するものであり、Bot側でカスタムEmbedを生成しない。

したがって、本文のMarkdownとリンク先は固定できるが、プレビューの画像・説明文・表示タイミングはDiscordまたはAtCoder側の仕様に依存する。Discord APIリクエストではリンクプレビューを抑制するフラグを指定しない。

3.4 開始・終了時刻

画像の本文に終了時刻が必要なため、ContestモデルにendTimeを追加する。AtCoder一覧から開始時刻と開催時間を取得し、次の式で算出する。

```
endTime = startTime + duration
```

開催時間を確実に取得できない場合は、推測値やABC固有の固定値で補完せず、候補を無効として通知しない。開始・終了時刻の双方が検証できた場合だけ通知本文を生成する。

3.5 終了後の次回開催日時通知

保存済みABCのendTimeに到達した最初の30秒周期のループでAtCoder一覧を取得する。次回の未来ABCを確認できた場合は、次の本文を送る。日付・時刻は日本時間で表示し、終了日が異なる場合は終了側にも日付を付ける。

```
# **次回のAtCoder Beginner Contest {N}**

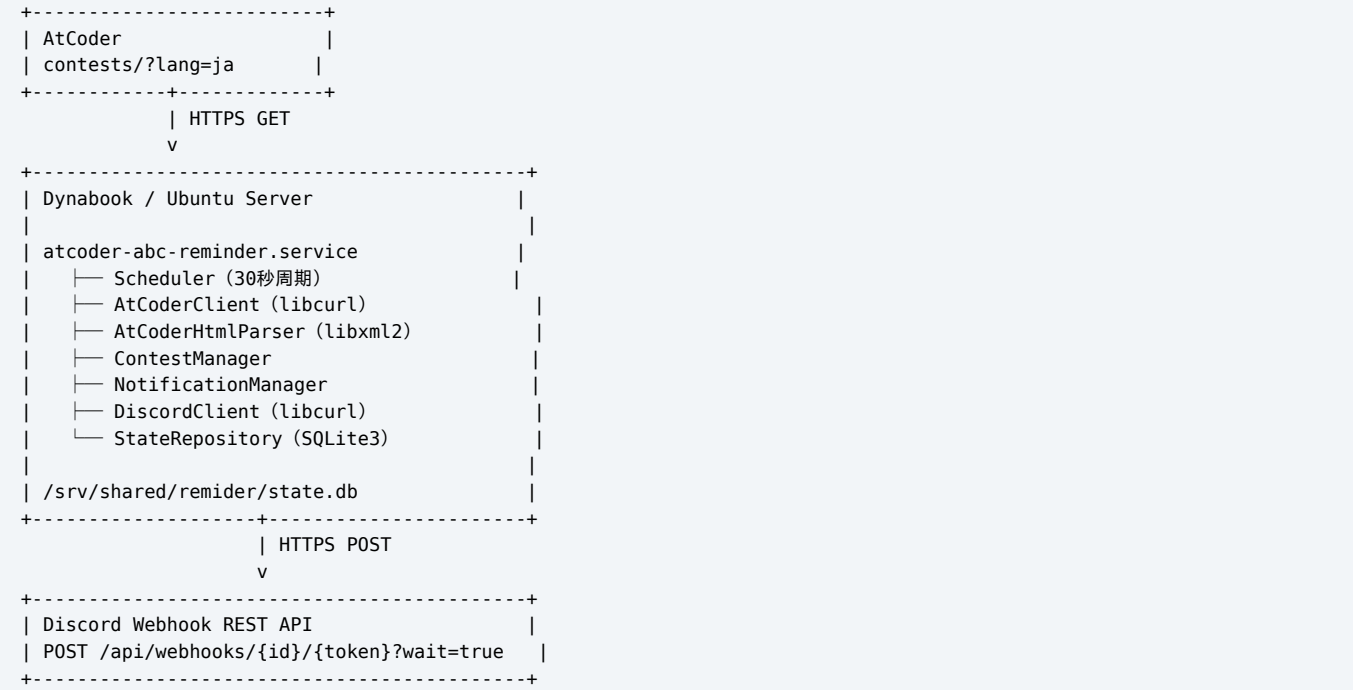
**次回は {YYYY/MM/DD START} ～ {END}（日本時間）に [AtCoder Beginner Contest {N}](https://atcoder.jp/contests/abc{N}) が開催される予定です。**
```

この通知はロールメンションを含めず、allowed_mentions.parseとallowed_mentions.rolesをともに空配列にする。開始2時間前・日時変更・土曜未開催の通知では従来どおり設定済みロールをメンションする。

次回が未発表、一覧構造が不明、またはABC行が不完全な場合は日時を推測せず再取得する。次回への移行と送信待ち通知を同一SQLiteトランザクションで保存し、再起動後も未送信なら再試行する。成功済みの通知は再送しない。長期停止中に通知対象の開始時刻を過ぎた場合は、その保留通知をskippedとして古い予定を送らない。

保留通知の再試行時もAtCoder一覧を再取得して対象IDを確認し、変更された日時を送信前にSQLiteへ反映する。対象を確認できない場合は通知を保留する。確認できた日時が延期されている場合は、その新しい予定を使用する。

4. システム構成



Ubuntu Server側でネットワーク接続状態を確認する機能は実装しない。HTTP要求が失敗した場合のみ、その要求の失敗として処理し、保存状態を維持してバックオフ後に再試行する。

5. 実行環境・使用技術

5.1 対象環境

- Ubuntu Server LTSを搭載したDynabook

- x86_64 Linuxを第一対象とする

- C++17

- CMake

- systemd

- OpenSSL/TLS（libcurl経由）

Ubuntuの具体的なバージョンは実装時に対象機体のLTSへ固定する。ホストのタイムゾーン設定に依存しないため、アプリケーション内部では常にJSTを明示して扱う。

5.2 使用ライブラリ

ライブラリ	用途	採用理由
libcurl	HTTPS GET/POST、タイムアウト、HTTPヘッダー取得	Linuxで標準的で、TLSを任せられる
libxml2	不完全HTMLを含むAtCoderページ解析	正規表現のみより誤解析しにくい
SQLite3	通知状態・現在コンテストの永続化	単一ファイル、トランザクション、無料
nlohmann/json	Discord JSON本文の生成・レスポンス解析	JSONエスケープと解析を手作業にしない
OpenSSL	TLS基盤	libcurlのHTTPS検証に使用

依存関係はOSパッケージとして導入し、独自の常駐サーバーや有料SDKは追加しない。標準ライブラリだけで安全に処理できる箇所は標準ライブラリを使用する。

5.3 開発・配置ディレクトリ

```
atcoder-abc-reminder/
├── CMakeLists.txt
├── README.md
├── .gitignore
├── include/
│   └── atcoder_reminder/
│       ├── atcoder_client.hpp
│       ├── atcoder_html_parser.hpp
│       ├── config.hpp
│       ├── discord_client.hpp
│       ├── domain.hpp
│       ├── http_client.hpp
│       ├── message_template.hpp
│       ├── ports.hpp
│       ├── reminder_policy.hpp
│       ├── state_repository.hpp
│       └── time_service.hpp
├── src/
│   ├── atcoder_client.cpp
│   ├── atcoder_html_parser.cpp
│   ├── discord_client.cpp
│   ├── http_client.cpp
│   ├── main.cpp
│   ├── message_template.cpp
│   ├── reminder_policy.cpp
│   ├── state_repository.cpp
│   └── time_service.cpp
├── tests/
│   └── test_main.cpp
└── deploy/
    ├── atcoder-abc-reminder.env.example
    └── atcoder-abc-reminder.service
```

5.4 デプロイ成果物

サーバーへ配置するものは次に限定する。

成果物	配置先	備考
実行ファイル	/srv/shared/remider/bin/atcoder-abc-reminder	CMakeのReleaseビルド
SQLite	/srv/shared/remider/state.db	初回起動時に生成。既存DBを上書きしない
リリース情報	/srv/shared/remider/release-info.txt	バージョン、ビルド日時、Git SHA等

成果物	配置先	備考
systemd定義	/etc/systemd/system/atcoder-abc-reminder.service	SSH先でsudoにより更新
秘密情報	/etc/atcoder-abc-reminder/atcoder-abc-reminder.env	ソース・共有フォルダーへコピーしない

サーバー上の/srv/shared/remiderは実行環境として扱い、ソースコードの編集場所にはしない。デプロイは次の順序で行う。

1. 開発環境でテスト、静的解析、Releaseビルドを実行する。

2. 実行ファイルのSHA-256を計算する。

3. SSH経由で/srv/shared/remider/.staging/へ転送する。

4. サーバー上でハッシュ値、所有者、実行権限を確認する。

5. サービスを停止する。

6. 既存実行ファイルをバックアップしてから新しい実行ファイルへ切り替える。

7. systemctl daemon-reload後にサービスを起動する。

8. systemctl is-activeとjournalで起動結果を確認する。

SQLiteのstate.dbは通常のデプロイで上書きしない。スキーマ変更が必要な場合は、バックアップ取得と互換性確認を行ったうえで、アプリケーションのマイグレーション処理を使用する。

6. モジュール設計

6.1 domain

外部ライブラリに依存しない業務ルールを担当する。

```
namespace reminder {

using UnixSeconds = std::int64_t;

struct Contest {
    std::string id;       // abc475
    int number{};
    UnixSeconds startTime{};
    UnixSeconds endTime{};
    std::string url;      // 検証済みの正規URL
};

enum class NotificationKind {
    Normal,
    ScheduleChanged,
};

enum class NotificationStatus {
    None,
    Pending,
    Sent,
};

struct CurrentContest {
    Contest contest;
    UnixSeconds reminderTime{};
    NotificationStatus normalStatus{NotificationStatus::Pending};
    std::optional<UnixSeconds> normalSentForStartTime;
    NotificationStatus changeStatus{NotificationStatus::None};
    std::optional<UnixSeconds> changeSentForStartTime;
    std::optional<NotificationKind> pendingKind;
    std::string pendingMessageKey;
```

```
};

} // namespace reminder
```

IDはabcに数字が1つ以上続く形だけを許可する。URLはIDから生成し、AtCoder以外のホストへ向けない。

6.2 time_service

- std::chrono::system_clockから現在のUnix秒を取得する。

- chronyまたはsystemd-timesyncd等、Ubuntu側の時刻同期を前提とする。

- アプリケーション自身はNTPサーバーへ直接接続しない。

- 現在時刻の妥当性を起動時に確認する。

- 表示時だけJSTへ変換する。

Ubuntuのシステム時刻が未同期と判断される場合は、通知とAtCoder候補確定を停止する。ただしWi-Fi接続状態を取得・監視する処理は持たない。

6.3 atcoder_client

https://atcoder.jp/contests/?lang=jaへlibcurlでGETする。

- 接続タイムアウト: 30秒

- 転送タイムアウト: 30秒

- リダイレクト: 最大3回

- 最終ホスト: atcoder.jpだけを許可

- HTTP 2xx以外: 取得失敗

- レスポンス上限: 2MiB

- TLS証明書検証: 有効

ESP32向けの256KiB制限は廃止するが、無制限読込は行わない。上限超過・取得失敗・解析失敗時には、現在のSQLite状態を更新しない。

6.4 atcoder_html_parser

libxml2のHTMLパーサーを使用し、AtCoderページをHTML文書として解析する。外部ネットワーク参照は禁止し、取得済み本文だけを入力にする。

処理手順:

1. HTMLをlibxml2で解析する。

2. コンテスト一覧の行またはリンク要素から/contests/abc[0-9]+を探す。

3. 同じコンテスト行から開始日時と開催時間を取得する。

4. Contestを生成し、ID、番号、URL、開始・終了時刻を検証する。

5. 同じIDが複数ある場合は、全フィールドが一致するものだけ重複排除する。一致しない場合はそのIDを無効にする。

6. 現在時刻より未来で、開始時刻より終了時刻が後の候補だけを残す。

7. 開始時刻昇順、ID昇順でソートする。

リンクの名称はABC判定に使用しない。スポンサー名付きの名称でもURLがABC形式なら採用する。

日時は、予定コンテスト表（#contest-table-upcoming）の各行にある<time class="fixtime fixtime-full">要素のテキストから取得する。現在確認できる形式はYYYY-MM-DD HH:MM:SS+0900である。開始日時、同じ行の開催時間（例:01:40）

、コンテストリンクを対応付け、endTime = startTime + durationで終了時刻を算出する。日時・開催時間のどちらかが抽出できない場合は候補を破棄する。HTML構造が変わり、候補の妥当性を確定できない場合は解析エラーとする。

6.5 contest_manager

取得済み候補から次回ABCを選択し、現在状態との差分を適用する。

```
候補なし（一覧構造を確認できない） -> 取得失敗扱いで状態維持・通知保留
新しいABC                       -> normal=pendingで新規保存
同一ABC・日時変更なし           -> 状態を維持
同一ABC・通知前に日時変更       -> 開始/終了/通知時刻を更新
同一ABC・通知後に日時変更       -> 訂正通知をpendingにする
保存済みABCが未来なのに一覧から消失 -> 状態を維持し、その回の通知を保留
現在ABCの開始時刻を経過         -> 終了時刻まで現在状態を保持、通常通知は送らない
現在ABCの終了時刻を経過         -> 次回候補と終了後通知を同一トランザクションで保存
```

通常リマインダーでは、予定表を取得できても完全な未来ABC候補が0件の場合は取得失敗扱いとする。現在状態を削除・置換せず、バックオフ後に再取得する。土曜確認に限り、予定表構造とABC行を正常に解析できた空の一覧も確認結果として扱うが、通常リマインダーの保存状態は変更しない。

土曜確認は#contest-table-upcoming、#contest-table-action、#contest-table-recentの開始日をJSTで比較する。ABC行の日時・開催時間が不正、リンクが曖昧、または予定が競合する場合は未開催と判断せず、5分後に再確認する。

6.6 reminder_policy

```
constexpr std::int64_t kReminderBeforeSeconds = 2 * 60 * 60;

UnixSeconds reminderTime(const Contest& contest) {
    return contest.startTime - kReminderBeforeSeconds;
}

bool shouldSendNormal(const CurrentContest& state, UnixSeconds now) {
    return state.normalStatus == NotificationStatus::Pending
        && now >= state.reminderTime
        && now < state.contest.startTime;
}
```

開始時刻を過ぎてから初めて検出した通常通知は送らない。開始前に通知時刻を過ぎていれば、次回ループで即時送信する。

6.7 message_template

通常通知ではJST日時をHH:mmで整形し、終了後の次回通知では開始側にYYYY/MM/DD HH:mmを使用する。3章の固定本文を生成する。コンテスト番号は整数から生成し、AtCoder HTMLの名称を本文へ直接コピーしない。

DiscordのJSON本文はnlohmann/jsonで生成し、手作業の文字列連結によるJSON破壊を防ぐ。allowed_mentions.parseを空配列にしたうえで、設定された競プロロールIDだけをallowed_mentions.rolesへ指定する。

```
{
  "allowed_mentions": {
    "parse": [],
    "roles": ["{ROLE_ID}"]
  }
}
```

6.8 discord_client

使用エンドポイント:

```
POST https://discord.com/api/webhooks/{webhook_id}/{webhook_token}?wait=true
```

POST本文:

```
{
  "content": "...画像仕様の本文...",
  "allowed_mentions": { "parse": [], "roles": ["{ROLE_ID}"] },
  "username": "AtCoder ABC Reminder"
}
```

URLの埋め込みプレビューを表示させるため、Embed抑制フラグは送らない。

終了後の次回通知はsend(..., mentionRole=false)を使い、allowed_mentionsを{"parse": [], "roles": []}にする。

通知先チャンネルでWebhookを作成する。競プロロールはメンション可能に設定する。

7. 永続化設計

7.1 SQLite

保存先:

```
/srv/shared/remider/state.db
```

SQLiteをWALモードで使用し、状態更新をトランザクションで行う。DBファイル、WALファイル、SHMファイルを同一ディレクトリへ置く。

7.2 スキーマ

```
CREATE TABLE IF NOT EXISTS current_contest (
    singleton       INTEGER PRIMARY KEY CHECK (singleton = 1),
    contest_id      TEXT NOT NULL,
    contest_number  INTEGER NOT NULL,
    start_time      INTEGER NOT NULL,
    end_time        INTEGER NOT NULL,
    reminder_time   INTEGER NOT NULL,
    normal_status   TEXT NOT NULL CHECK (normal_status IN ('pending', 'sent')),
    normal_sent_at_start INTEGER,
    change_status   TEXT NOT NULL CHECK (change_status IN ('none', 'pending', 'sent')),
    change_sent_at_start INTEGER,
    pending_kind    TEXT CHECK (pending_kind IN ('normal', 'change')),
    pending_key     TEXT,
    last_fetched_at INTEGER,
    updated_at      INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS notification_history (
    message_key     TEXT PRIMARY KEY,
    contest_id      TEXT NOT NULL,
    kind            TEXT NOT NULL CHECK (kind IN ('normal', 'change')),
    start_time      INTEGER NOT NULL,
    status          TEXT NOT NULL CHECK (status IN ('pending', 'sent', 'skipped')),
    discord_message_id TEXT,
    updated_at      INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS weekly_absence_checks (
    check_date      TEXT PRIMARY KEY,
    completed_at    INTEGER NOT NULL
);

CREATE TABLE IF NOT EXISTS next_contest_notices (
    message_key     TEXT PRIMARY KEY,
    previous_contest_id TEXT NOT NULL,
    previous_start_time INTEGER NOT NULL,
    previous_end_time INTEGER NOT NULL,
    contest_id      TEXT NOT NULL,
    contest_number  INTEGER NOT NULL,
    start_time      INTEGER NOT NULL,
    end_time        INTEGER NOT NULL,
    status          TEXT NOT NULL CHECK (status IN ('pending', 'sent', 'skipped')),
    discord_message_id TEXT,
    updated_at      INTEGER NOT NULL
);
```

notification_historyはコンテストが次回へ移行した後も通知キーとDiscordメッセージIDを保持し、再起動後の重複防止と運用確認に使用する。 weekly_absence_checksはJSTの日付ごとに土曜確認の完了時刻を保存し、同日の確認・通知を重複させない。 next_contest_noticesは終了したABCごとにnext:{previous_contest_id}:{previous_start_time}をキーとし、次回開催日時通知を独立して管理する。既存DBにはこの表だけを追加し、従来の表・履歴を作り直さない。

7.3 通知状態の更新

通常通知の送信前に、同一トランザクションでnotification_history.status = 'pending'とcurrent_contest.pending_kind = 'normal'を保存する。Discord送信成功後に履歴と現在状態をsentへ更新する。

日時変更通知も同様にkind = 'change'の別キーで管理する。同一コンテストが複数回延期された場合は、変更後の開始時刻ごとに別キーを生成する。

終了後通知はadvanceToNextContestで新しいcurrent_contestとnext_contest_notices.status = 'pending'を同時に保存する。終了前の移行は拒否する。送信成功後は終了後通知だけをsentにし、次回大会の通常リマインダーはpendingのまま維持する。送信待ちが次回大会の開始時刻を過ぎた場合はskippedにする。通信・5xx・429の失敗や送信後の保存失敗は5分後、設定エラーは30分後に再試行する。Webhookとの原子的保存ができない制約はこの通知にも適用される。

保存成功前に送信済みと確定しない。SQLite更新に失敗した場合は、次回ループで送信前の状態を再確認する。

7.4 外部送信との整合性

Discord送信とSQLite更新は同一トランザクションではないため、理論上の完全なExactly-onceは実現できない。本設計では次を組み合わせる。

1. 送信前に保留状態を保存

2. WebhookのWait=true応答からDiscordメッセージIDを取得して保存

3. 再起動時もSQLiteの保留キーを確認して送信状態を復元

4. 通信結果が不明な場合は未送信扱いのまま再試行する

Webhookではチャンネル履歴を一覧取得できないため、通信結果が不明なタイミングでDiscord側だけに投稿が成立した場合は重複投稿の可能性が残る。

8. スケジュールと状態遷移

8.1 AtCoder確認間隔

次回ABCまでの残り時間	確認間隔
24時間超	60分
6時間超、24時間以下	15分
6時間以下、開始前	5分
対象なし	60分

起動直後、前回取得失敗後のバックオフ完了時、時刻同期完了時は即時取得する。通知時刻の判定は30秒周期の内部時計だけで実施し、AtCoderへアクセスしない。

現在ABCの終了時刻に到達した場合は、通常の確認間隔を待たずにAtCoderを即時取得する。終了後に候補がない場合は現在状態を保持して取得バックオフを続け、不完全な一覧に未来候補が残っている場合も次回と確定せず5分後に再確認する。

8.2 土曜日の確認スケジュール

dueSaturdayCheckDateはJSTの土曜日19:00以降に当日の日付キーを返す。完了済みでなければAtCoder一覧を即時取得する。予定・開催中・直近終了の一覧に当日開始のABCがあれば完了記録だけを保存し、当日分がなければDiscordへ通知する。重複抑止はSQLiteのJST日付キーで行い、Webhookへ照合キーやBot認証ヘッダーを送らない。予定・直近終了の表とその行構造を確認できない場合や不完全なABC行を検出した場合は、未開催と判断せず再確認する。開催中の表は大会がない場合に省略されるため必須にしない。通信失敗は取得バックオフ、構造やABC行の検証失敗は5分後に再試行する。

送信が成功した後にのみweekly_absence_checksへ完了を記録する。Discord送信・DB保存に失敗した場合はエラーを記録して再試行する。土曜19時以降にサービスが起動した場合も同じ判定を行う。

8.3 状態

```
STARTING
  |
  v
LOADING_STATE
  |
  v
FETCHING_CONTEST
  |--失敗--> WAIT（状態維持、バックオフ）
  |
  v
WAIT_FOR_REMINDER
  |--通知時刻到達--> SENDING_NORMAL
  |--終了時刻経過--> FETCHING_NEXT
  |--確認時刻到達--> FETCHING_CONTEST
  |
  +--日時変更後の訂正通知--> SENDING_CHANGE

SENDING_NORMAL/SENDING_CHANGE
  |--成功--> MARK_SENT -> WAIT_FOR_REMINDER
  |--失敗--> WAIT（保留状態を維持し、バックオフ後にPOSTを再試行）

FETCHING_NEXT --取得成功--> 新しいABCと終了後通知を保存 -> SENDING_NEXT（メンションなし）
FETCHING_NEXT --取得失敗--> 現在状態を維持 -> WAIT
SENDING_NEXT --成功--> 終了後通知だけsent -> WAIT_FOR_REMINDER
SENDING_NEXT --失敗--> pendingを維持 -> バックオフ後に再試行

土曜19時以降かつ当日確認未完了
  -> FETCHING_CONTEST
       |--当日ABCあり--> 保存済みとして完了
       |--当日ABCなし--> SENDING_NO_ABC -> 保存済みとして完了
       |--取得/送信失敗--> 完了記録なし -> バックオフ後に再試行
```

8.4 起動時

1. systemdからプロセスを起動する。

2. SQLiteを開き、スキーマを検証する。

3. SQLiteの保留通知（終了後通知を含む）と送信済み状態を復元する。Discord履歴への照会は行わない。

4. OS時刻が有効であることを確認する。

5. AtCoder一覧を即時取得する。

6. 現在状態と候補を比較し、日時変更または次回移行を適用する。

7. 30秒周期のループを開始する。

9. エラー処理

9.1 ネットワーク・HTTP

Wi-Fi接続確認は行わないが、HTTP要求の失敗は処理する。

エラー	処理
DNS、接続、TLS、タイムアウト	5分後から指数バックオフで再試行、最大1時間
AtCoder HTTP 5xx	保存状態を維持して再試行
AtCoder HTTP 4xx	URL・アクセス仕様をログし、30分以上の間隔で再試行
Discord 429	Retry-After以上待機して再試行
Discord 5xx	指数バックオフで再試行

エラー	処理
Discord 400	本文・リクエスト不正。未送信状態を保持し、30分間隔でログ
Discord 401/403	Token・チャンネル・権限の問題。未送信状態を保持し、30分間隔でログ
HTML解析エラー	現在状態を更新せず、次回取得
SQLiteエラー	通知済みにせず、エラー終了または次回処理へ回す

9.2 プロセス障害

- 最上位で例外を捕捉し、秘密情報を含めずにjournalへ出力する。

- 予期しない終了時はsystemdのRestart=on-failureで再起動する。

- 同時実行を防ぐため、systemdのサービスを1インスタンスに限定する。

- SQLiteのトランザクション途中で終了しても、SQLiteのロールバックに任せる。

10. セキュリティ設計

10.1 秘密情報

Discord Webhook URLはソースコードやGitへ記録しない。Ubuntu上のroot所有ファイルへ保存する。

```
/etc/atcoder-abc-reminder/atcoder-abc-reminder.env
```

権限:

```
root:root 600
```

systemdがroot権限でEnvironmentFileを読み込み、サービスプロセスへ渡す。サービス専用ユーザーへファイルの直接読み取り権限を付与しない。Webhook URLをコマンドライン引数に置かない。

10.2 systemdサンドボックス

サービス定義では次を設定する。

```
User=abc-reminder
Group=abc-reminder
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
ProtectSystem=strict
ReadWritePaths=/srv/shared/remider
Restart=on-failure
RestartSec=30
```

ネットワーク通信に必要な権限以外を付与しない。ログへToken、Password、Authorizationヘッダー、HTML本文を出力しない。

10.3 入力検証

- AtCoder URLはhttps://atcoder.jp/contests/abc[0-9]+に限定する。

- ABC番号は整数として解析し、異常に大きい値を拒否する。

- 日時・開催時間が不正な候補は採用しない。

- Discord本文は固定テンプレートと検証済み値だけで生成する。

- allowed_mentions.parseを空配列にし、競プロロールIDだけをallowed_mentions.rolesへ指定する。

- 外部HTMLから取得した文字列をSQLへ連結せず、プレースホルダーを使う。

11. systemd配置設計

サービスファイル例:

```
[Unit]
Description=AtCoder ABC Discord Reminder Bot
After=network-online.target

[Service]
Type=simple
ExecStart=/srv/shared/remider/bin/atcoder-abc-reminder
EnvironmentFile=/etc/atcoder-abc-reminder/atcoder-abc-reminder.env
User=abc-reminder
Group=abc-reminder
WorkingDirectory=/srv/shared/remider
Restart=on-failure
RestartSec=30
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
ProtectSystem=strict
ReadWritePaths=/srv/shared/remider

[Install]
WantedBy=multi-user.target
```

network-online.targetは起動順序の宣言にのみ使用する。アプリケーションはWi-Fi接続確認を行わず、HTTPの成否で通信可能性を判断する。

Dynabookを通知装置として常時稼働させるため、Ubuntuの自動サスペンド・休止を無効にする設定を導入手順へ含める。これはアプリケーションのWi-Fi判定とは別の、機体の電源管理設定である。

12. テスト設計

12.1 単体テスト

GoogleTest等の無料テストフレームワーク、またはCMakeで導入できる最小テストランナーを使用する。

対象	確認内容
URL判定	abc475を採用し、arc475、名称だけのABC、別ホストを除外
HTML解析	正常、スポンサー名、重複、日時欠落、構造変更、巨大本文
日時	JST、ISO形式、無効日付、日跨ぎ、終了時刻計算
次回選択	過去・現在・未来・複数候補・同時刻
通知時刻	開始7200秒前、24時間境界、6時間境界
本文	画像仕様の改行、番号、開始・終了時刻、URL、絵文字
太字表示	通常通知と未開催通知の全表示文言にDiscord Markdownの太字を適用
土曜日確認時刻	JST土曜18:59は対象外、19:00以降は当日確認日を返す
土曜開催判定	未来・開催中・当日終了のABC、翌日のABC、空予定表、不正なABC行を区別
終了後通知	開始時には移行せず、終了時刻で次回を選択。日時・URL表示とメンション禁止
終了後通知の永続化	既存DBへの表追加、移行と保留保存の原子性、再起動復元、重複抑止、日時更新、期限切れ
変更前	通常通知未送信時に通知時刻だけ再計算
変更後	訂正通知を1回、再変更時は新日時ごとに1回
SQLite	トランザクション、再起動復元、保留・送信済み状態

対象	確認内容
Discord	2xx、429、5xx、タイムアウト、Webhook応答、JSONエスケープ

12.2 統合テスト

libcurl、Discord、AtCoderをFakeサーバーへ差し替え、次を実行する。

- 起動から次回ABC取得・保存

- 通知時刻到達からDiscord POST・SQLite更新

- POSTタイムアウト後も保留状態を維持し、再試行する

- AtCoder取得失敗後も既存の通知時刻を維持

- 通知前の日時変更

- 通知後の訂正通知

- プロセス再起動後のSQLite保留通知復元と再試行

- コンテスト終了後の次回ABC移行とメンションなし通知、通常リマインダーの独立維持

- 連続HTTP失敗からのバックオフ復旧

12.3 実機・運用テスト

Dynabook上で次を確認する。

- systemctl enable --nowで自動起動する

- 24時間以上連続稼働する

- systemdによる異常終了後の自動再起動

- Ubuntu再起動後のSQLite状態復元

- 自動サスペンド・休止無効化後に通知時刻まで稼働する

- AtCoder実ページから開始・終了時刻を取得する

- Discordの本文とリンクプレビューが画像仕様になる

- 通知後に開催日時を変更した場合、訂正通知が送られる

- 土曜日19時に当日のABC有無を判定し、未開催時だけ通知する

- 土曜未開催通知を再起動後も同じ日に重複送信しない

- 通常通知と土曜未開催通知がDiscord上で太字になる

13. 要件トレーサビリティ

要件	設計対応
FR-01 Wi-Fi接続	変更によりアプリケーションでの接続確認を対象外とする。OSのネットワークを利用する。
FR-02 時刻取得	Ubuntuの時刻同期とC++時刻サービス、JST変換
FR-03 AtCoder取得	libcurlによるHTTPS GETとlibxml2解析
FR-04 次回ABC	ContestManagerの未来候補ソート
FR-05 2時間前	startTime - 7200
FR-06 Discord通知	Discord REST API POST
FR-07 通知内容	固定本文、太字のDiscord Markdown、AtCoder URL、標準リンクプレビュー
FR-08 日時変更	同一IDの開始・終了時刻比較
FR-09 通知後変更	notification_historyと訂正通知
FR-10 重複防止	SQLite、Webhook応答のメッセージID保存
FR-11 状態保存	SQLiteのローカルファイルとトランザクション
FR-12 次回移行	終了時刻以降に取得・検証できた次回候補へ置換
FR-13 土曜確認	JST土曜19時以降に当日ABCを確認し、未開催なら太字で通知
FR-14 終了後通知	終了後の次回日時通知をメンションなしで送信し、保留・完了状態を保存
追加: 土曜未開催通知	JST土曜19時判定、当日のABC抽出、weekly_absence_checksによる重複防止
NFR-01 軽量性	ESP32制約を解除。ただし本文・候補・HTTP応答に上限を設ける
NFR-02 常時稼働	systemd、自動再起動、タイムアウト
NFR-03 再起動耐性	SQLite、保留通知
NFR-04 障害耐性	HTTP/Discord/HTML/SQLite別の失敗処理
NFR-05 負荷抑制	60分/15分/5分の段階的確認間隔

FR-01はユーザー指定により「Wi-Fi接続状態を確認しない」へ変更した。Wi-Fiが利用できるかどうかを検出する処理は実装しないが、HTTP要求自体の失敗は通知保護のため処理する。

14. 費用・運用

14.1 継続費

- C++、CMake、libcurl、libxml2、SQLite3、nlohmann/json、systemd: 無料で利用できるソフトウェアを使用する。

- AtCoder: 公開ページを取得するだけで、有料APIを利用しない。

- Discord: 既存サーバーの通知先チャンネルに作成したWebhookを利用し、Gateway常駐基盤や有料ホスティングを使わない。

- クラウドサーバー、DB、監視サービス: 使用しない。

月額固定費は増加させない。Dynabookの購入費や必要な周辺機器は一括購入費であり、電気代は機体を常時稼働させることによる変動費として発生する。

14.2 バックアップ

SQLiteファイルは通知履歴を含むため、サービス停止後にstate.db、state.db-wal、state.db-shmを整合した状態でバックアップする。バックアップ先は同一ディスク上だけにせず、必要に応じてユーザーが手動で外部媒体へコピーする。

15. 作業単位

UOW-01 C++ビルド骨格

- CMake、C++17、libcurl、libxml2、SQLite3、JSONライブラリを導入する。

- main.cppから起動・終了ログを出す。

- 完了条件: Ubuntu Server上でビルドと起動ができる。

UOW-02 ドメイン・時刻・本文

- Contest、通知時刻、終了時刻、画像仕様の本文を実装する。

- 完了条件: 日時境界と本文比較テストに通る。

UOW-03 AtCoder取得・解析

- libcurlとlibxml2で開始・終了時刻付きABC候補を取得する。

- 完了条件: フィクスチャと実ページで妥当な候補だけを抽出する。

UOW-04 SQLite永続化

- スキーマ、トランザクション、通知履歴、再起動復元を実装する。

- 完了条件: 送信前・送信後・保存失敗のテストに通る。

UOW-05 Discord通知

- Webhook REST POST、429、再試行、リンクプレビューを実装する。

- 完了条件: Discordの実チャンネルで本文とプレビューを確認する。

UOW-06 systemd・長期稼働

- サービスユーザー、環境ファイル、systemd、自動再起動、電源管理を設定する。

- 完了条件: Ubuntu再起動・プロセス異常終了・24時間稼働テストに通る。

16. 既知の制約・残存リスク

1. AtCoder一覧HTMLの構造変更時は候補を安全側で無効にする。実装更新が必要になる。

2. Discord送信とSQLite保存は別システムのため、数学的なExactly-onceは保証できない。Webhookでは通信結果不明時に重複投稿の可能性が残る。

3. Discordのリンクプレビューは外部サービスの生成結果であり、画像・説明・表示の完全一致は保証できない。本文とURL配置は画像仕様に固定する。

4. Dynabookが電源断、休止、故障した時間帯は通知できない。常時給電と自動サスペンド無効化を運用条件とする。

5. Webhook URLが漏洩した場合はDiscordチャンネル設定でWebhookを再生成し、環境ファイルを更新する。

6. Ubuntuの時刻同期が壊れている場合は誤通知防止のため通知しない。

17. 完成条件

- Ubuntu Server搭載Dynabook上でC++17プログラムが常駐する。

- AtCoderからABC番号、開始時刻、終了時刻を自動取得する。

- ABC番号の手動入力が不要である。

- 開始2時間前に画像仕様の本文をDiscordへ送信する。

- 通常通知と未開催通知の全表示文言を太字で表示する。

- 毎週土曜日19時（JST）に当日のABC開催予定を確認し、ない場合にDiscord通知を送信する。

- 土曜の確認結果をSQLiteへ保存し、同日の重複通知を防ぐ。

- 本文中のAtCoderインラインリンクにより標準リンクプレビューを表示できる。

- 開催日時変更を検知し、通知前は予定時刻を更新、通知後は訂正通知を行う。

- SQLiteで送信済みと記録した通常・訂正通知、および完了済みの土曜確認を再起動後も再送しない。送信結果が不明な場合の重複可能性は16章の制約に従う。

- Ubuntu・プロセス再起動後もSQLiteから状態を復元する。

- コンテスト終了後、次回ABCへ自動移行する。

- HTTP障害、HTML解析失敗、Discord失敗で誤った状態へ遷移しない。

- Discord Gateway、クラウドサーバー、月額サービスを必要としない。